

AI4LAM FANTASTIC FUTURES 2026

Implementing Secure, On-Premise Retrieval Augmented Generation

For Mission Archives and Custom Datasets

Gregory Ciccarelli, Ph.D.

Center for Astrophysics | Harvard & Smithsonian

2026-09-15

Retrieval Augmented Generation (RAG)

RAG: A chatbot that leverages external knowledge
(data outside the training dataset)

Retrieval Augmented Generation (RAG)

RAG: A chatbot that leverages external knowledge
(data outside the training dataset)

Workshop goals

1. Understand a RAG bot's purpose
2. Build a RAG bot and understand its components

GenAI Chatbots vs RAG Chatbots

- Generative AI: Text based word completion by a LLM
 - e.g. The capital of France is _____.
 - *During training*, 'pre load' with all country capitals
 - Answer is 'baked in' (if it exists at all)
- RAG: Generative AI with a document collection
 - *During query*, 'pre load' with all country capitals
 - Then ask, 'The capital of France is _____.'

LLM: Large language model: a neural network that outputs a probabilistically likely sequence of words

RAG in Real Life

The screenshot shows a Google search interface with the query 'tell me about rag pipelines'. The search results are filtered to 'All'. The first result is an 'AI Overview' section. The text of the overview is annotated with three red arrows pointing to specific parts: the first arrow points to 'A Retrieval-Augmented Generation (RAG) pipeline is', the second arrow points to 'a system that combines the text generation power of a Large Language Model (LLM) with external, trusted sources to provide accurate, up-to-date answers.', and the third arrow points to 'Instead of relying only on the fixed information it learned during its initial training, an LLM uses a RAG pipeline to search private documents, company databases, or live files first, and then writes an answer based on those real facts.' To the right of the text is a diagram titled 'Basic RAG Pipeline' showing a flow from 'User Query' to 'Embedder' to 'Vector Store' to 'Retriever' to 'LLM' to 'Answer'. Below the diagram is a small table with 'Step 1: Embedding' and 'Step 2: Data Retrieval'. Below the text is a list of 'Why Use a RAG Pipeline?' with three bullet points: 'Reduces hallucinations', 'Saves money', and 'Adds trust'. Each bullet point is annotated with a red arrow pointing to it. The 'Adds trust' bullet point is also annotated with a red arrow pointing to the text 'The AI can cite specific sources and documents so you can check where the answer came from.'

Google

tell me about rag pipelines

AI Mode All Images Videos Shopping Short videos News More Tools

AI Overview

A Retrieval-Augmented Generation (RAG) pipeline is a system that combines the text generation power of a Large Language Model (LLM) with external, trusted sources to provide accurate, up-to-date answers. Skyvia

Instead of relying only on the fixed information it learned during its initial training, an LLM uses a RAG pipeline to search private documents, company databases, or live files first, and then writes an answer based on those real facts. www.nimbleway.com +1

Basic RAG Pipeline

User Query

Embedder

Vector Store

Retriever

LLM

Answer

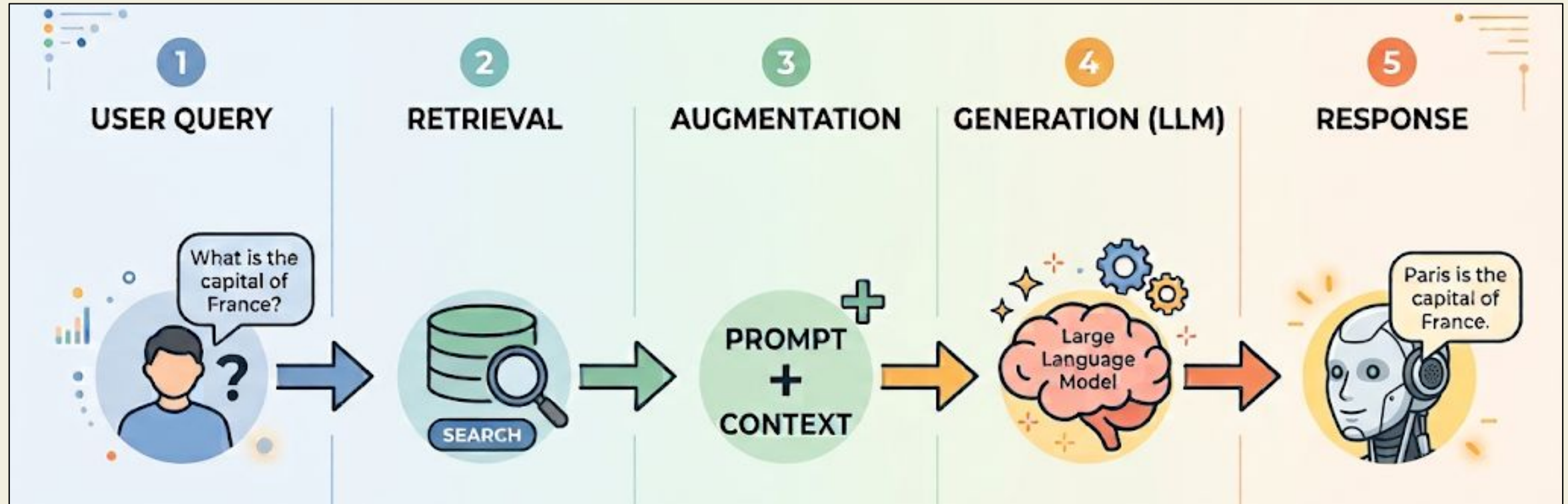
Step 1: Embedding

Step 2: Data Retrieval

Why Use a RAG Pipeline?

- **Reduces hallucinations:** It stops the AI from guessing or making up false facts.
- **Saves money:** You do not need to retrain or fine-tune the entire AI model when your data changes.
- **Adds trust:** The AI can cite specific sources and documents so you can check where the answer came from. Wikipedia +2

The RAG Pipeline



RAG in Real Life



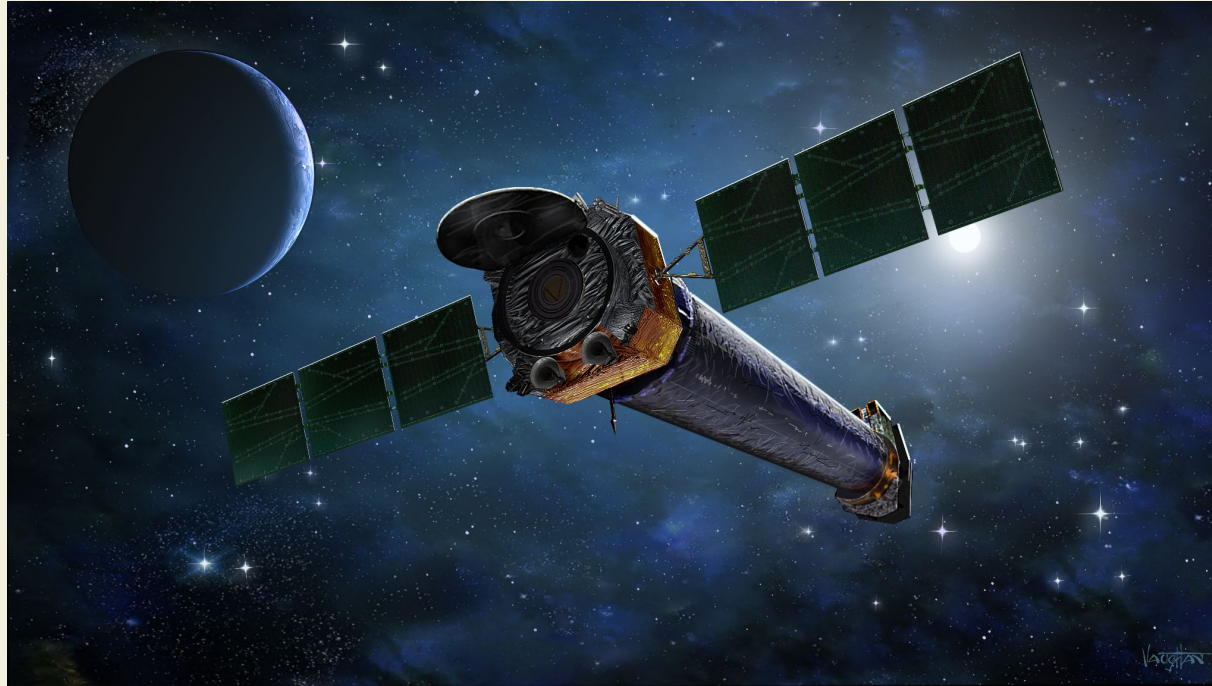
Google and OpenAI both provide ability to upload documents
for a 'no code' RAG experience

<https://notebook.google/>
[https://en.wikipedia.org/wiki/ChatGPT#/media/File:OpenAI_logo_2025_\(symbol\).svg](https://en.wikipedia.org/wiki/ChatGPT#/media/File:OpenAI_logo_2025_(symbol).svg)

Why Retrieval Augmented Generation?

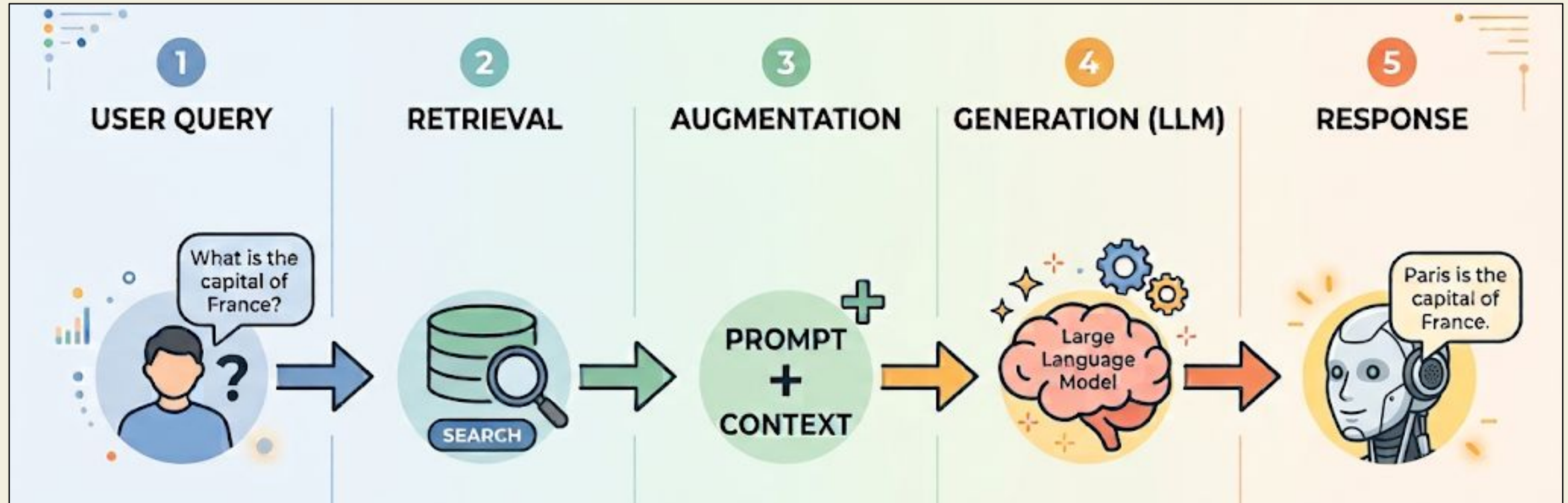
- Extend the AI's knowledge store
- Manage inference compute efficiently vs 'paste all'
- Leverage proprietary data without model re-training
- Avoid data egress from premises
- Reduce hallucination, save money, earn trust

RAG in Real Life: Chandra X-Ray Center



Chandra is the world's most powerful X-ray telescope
25+ years of archives: how to search abstracts?
Internal confluence wikis? flight notes? admin docs?

Workshop Outline



Module 1
Processing & Chunking

Module 2
Vector Stores

Module 3
Local LLMs and ChatApp

Module 4
Evaluation

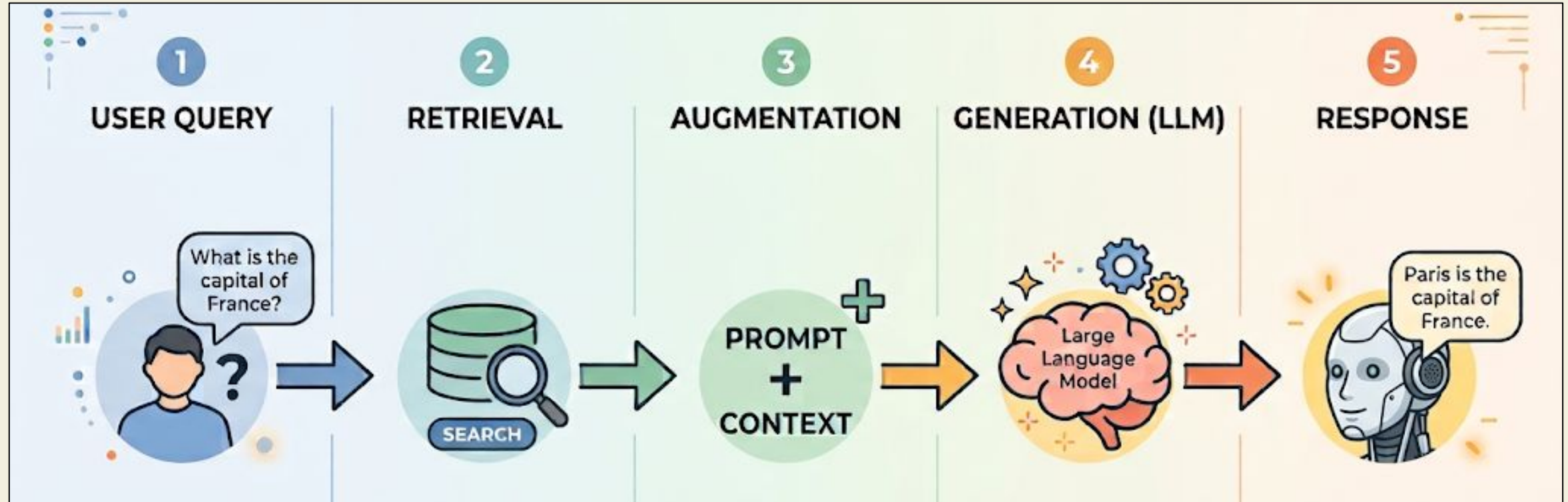
Workshop Structure

- Hands-on modules following RAG pipeline
- Each module
 - Short introduction
 - Self paced exercises
- Learning goals
 - Process and Chunk Technical Documentation
 - Generate Local Embeddings
 - Deploy Local LLMs
 - Evaluate Accuracy

Human Knowledge Expectations

- Familiar with Python
- Able to install software on your computer
- Tutorial stack
 - Python + packages
 - Ollama (LLM)
 - Visual studio code (or similar text editor)
- Maximizing workshop value
 - Read & understand the [README.md](#) files
 - Code is fun- but an LLM can solve the exercises
 - Use code and solutions for understanding not syntax

Module 1: Processing & Chunking



Module 1
Processing & Chunking

Module 2
Vector Stores

Module 3
Local LLMs and ChatApp

Module 4
Evaluation

Module 1: Processing & Chunking (P & C)

- P & C: prepare data sources for computers
- Processing
 - Convert data sources into plain text*
 - e.g. pdf, doc, docx, html $\Rightarrow$ text
- Chunking
 - Break data sources into pieces aka chunks
 - Retrieval is chunk based
 - Chunks are context for the LLM
 - Balance between too little and too much information

*RAG pipelines can be multi-modal (images, audio, video).
Multi-modal RAG is outside the scope of the workshop.

Compute Environment Setup

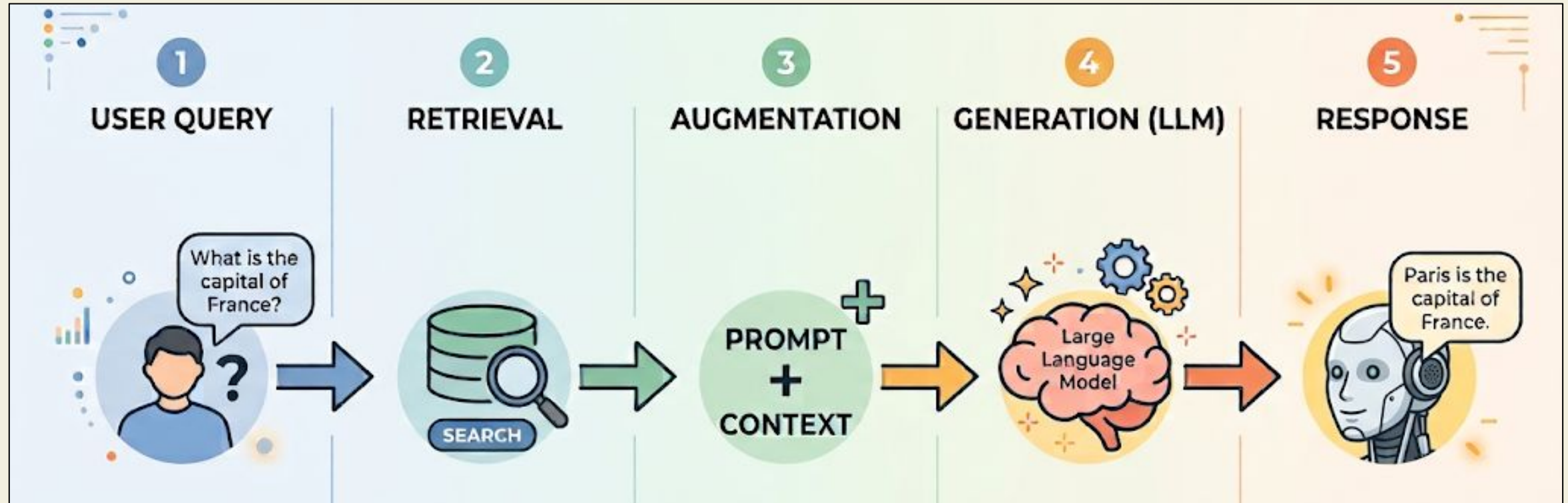
Step	Action	Command / Requirement
1. Repository	Clone the tutorial repo	git clone https://github.com/Smithsonian/ai4lam2026-workshop-rag
2. Environment	Create python environment	cd rag-ai4lam-2026 python3 -m venv .venv source .venv/bin/activate pip install -r requirements.txt
3. Models	Pull local LLM and Embedder	# install ollama: https://ollama.com/download ollama serve ollama pull gemma4:e4b ollama pull embeddinggemma

<https://github.com/Smithsonian/ai4lam2026-workshop-rag/README.md>

Module 1: Hands on

- Begin!
 - Read: module_01_data_parsing/[README.md](#)
 - Try: module_01_data_parsing/parsing_ex.py
 - Try: module_01_data_parsing/parsing_sln.py
- Reconvene in 15 minutes
- Feel free to work ahead
- Questions? Ask:
 - Search engines & chatbots
 - Your neighbor
 - Me “Greg”

Module 2: Vector Stores & Embeddings



Module 1
Processing & Chunking

Module 2
Vector Stores

Module 3
Local LLMs and ChatApp

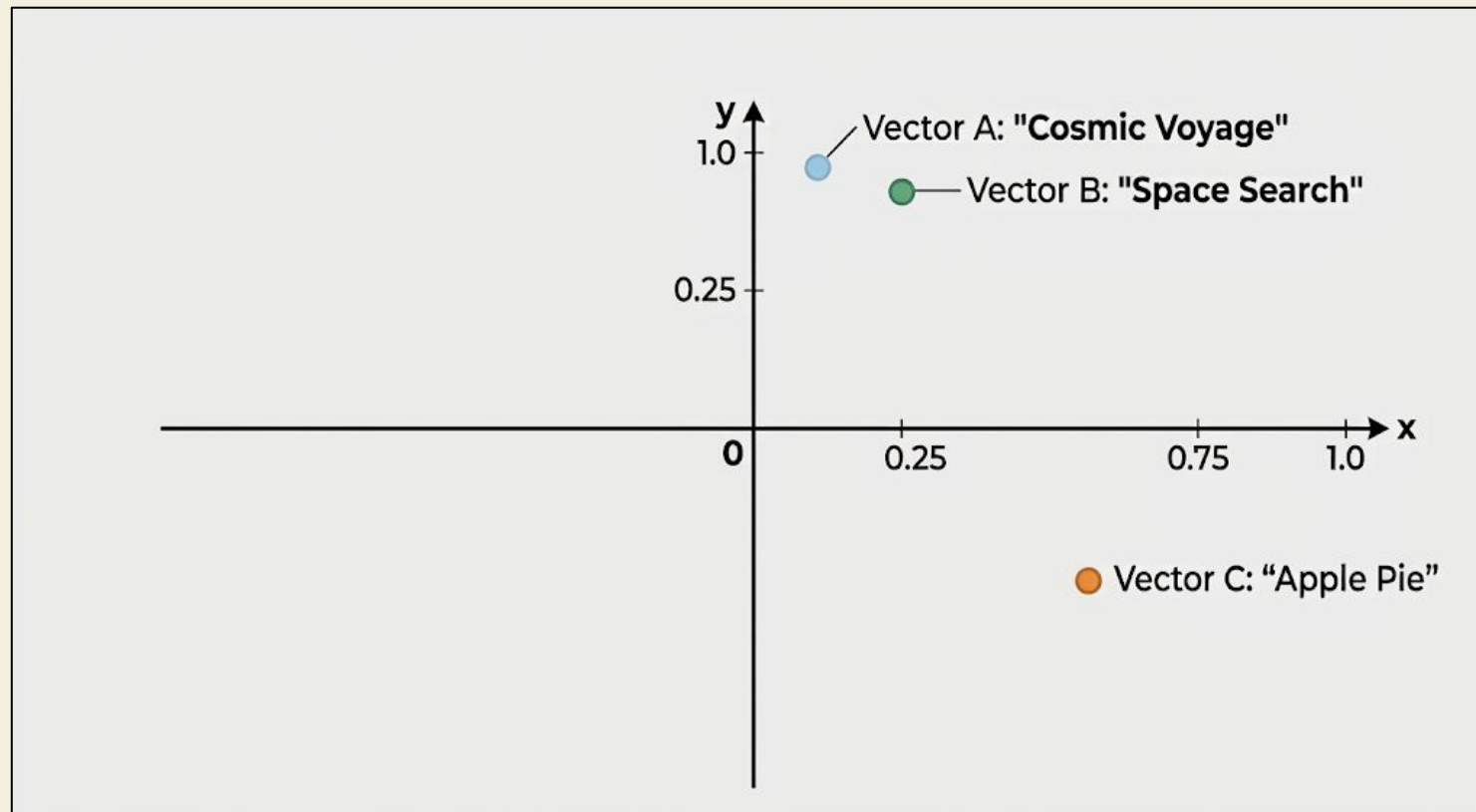
Module 4
Evaluation

Module 2: Vector Stores & Embedding

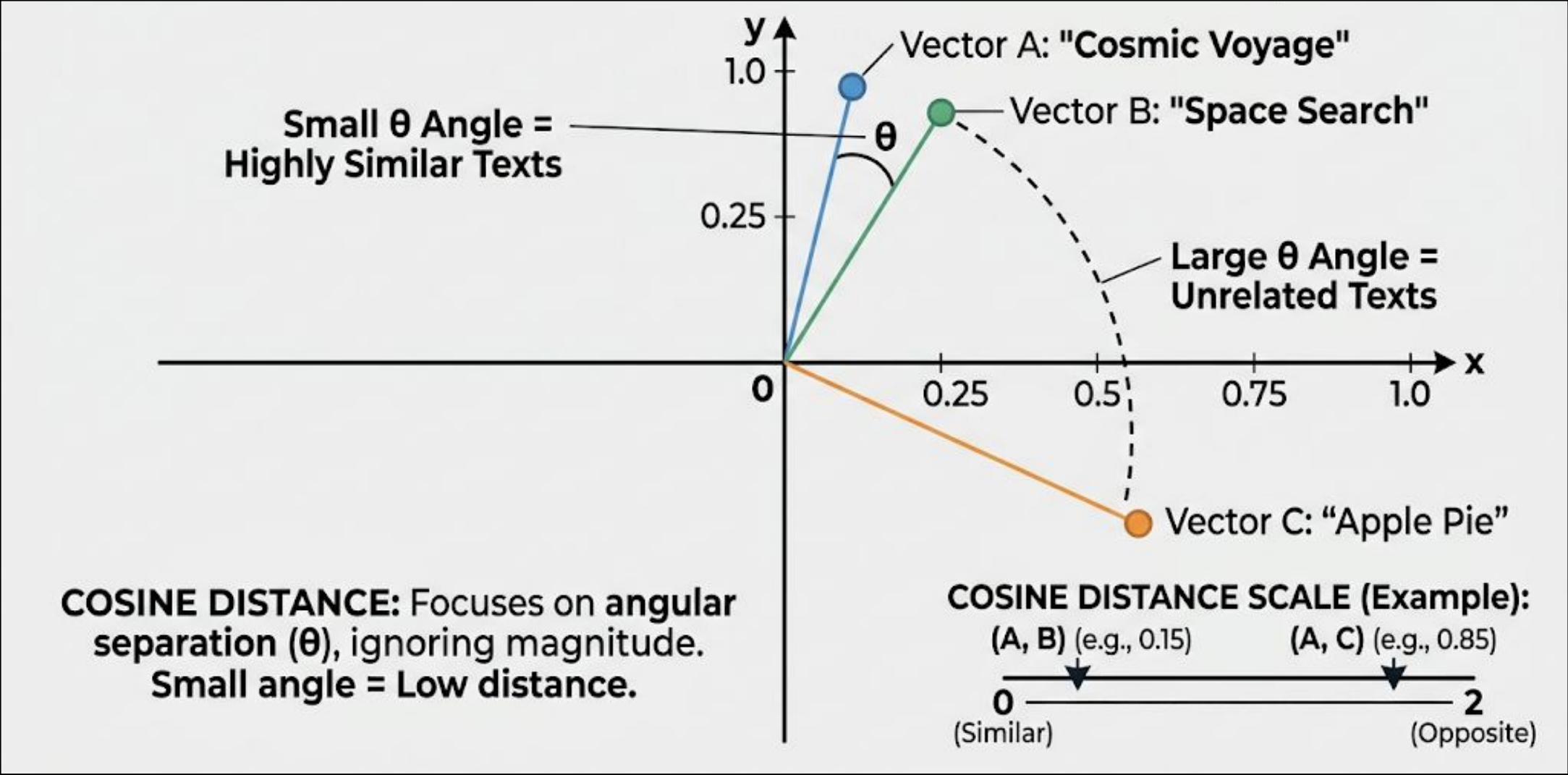
- Q: For a query, how to find *relevant* chunks of text?
- A: Represent text as list of numbers and do math
- Key Concepts
 - Vector: A list of numbers
 - Embedding: A vector representing a text's meaning
 - Vector Store/Vector DB: A database of embeddings
 - Traditional db, primary key = 1234
 - Vector db, primary key = [1, 57, 222, 64]
 - Vector keys support fast similarity retrieval
 - e.g. ChromaDB

Embeddings: Lists of Numbers

- Q: How to decide which numbers represent text?
- A: Use an embedding model
 - LLM converts text to numbers instead of text to text

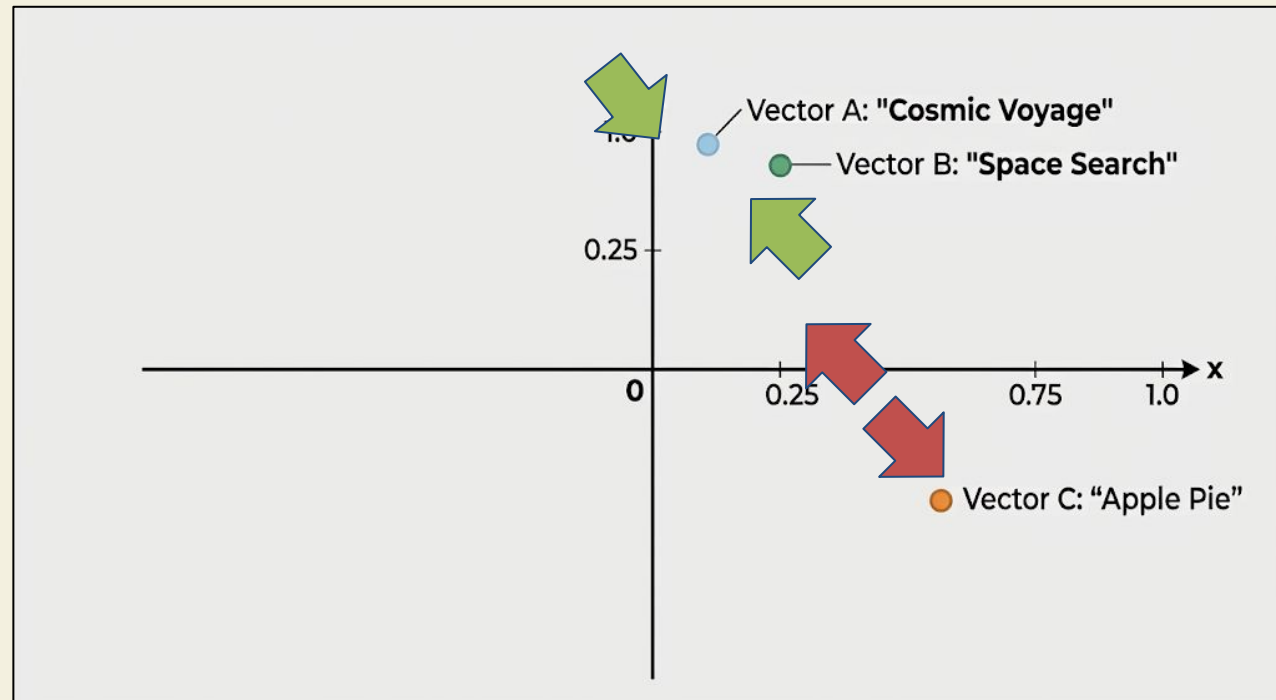


Embeddings & Text Distance



Where do Embedding Numbers Come From?

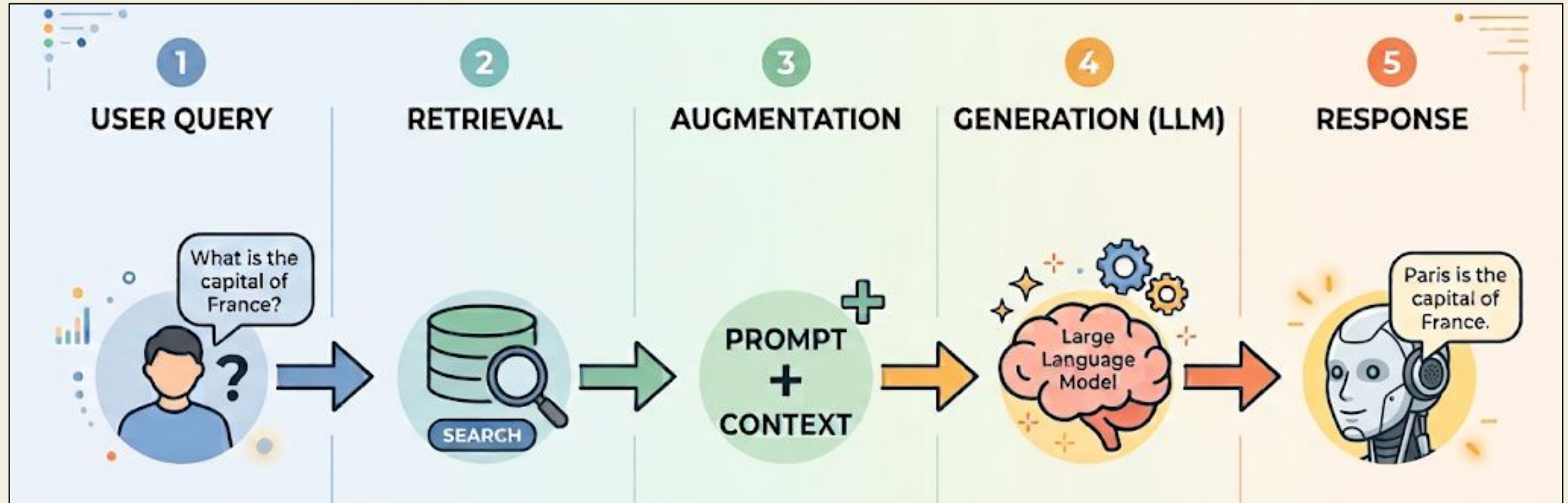
- If numbers represent text, who decides the numbers?
 - Start with random text to numbers assignment
 - Change numbers so 'similar text' from a 'ground truth' dataset have 'similar numbers'.



Module 2: Hands on

- Begin!
 - Read: module_02_vector_storage/[README.md](#)
 - Try: exercises
- Reconvene in 15 minutes
- Feel free to work ahead
- Questions? Ask:
 - Search engines & chatbots
 - Your neighbor
 - Me “Greg”

Module 3: RAG Bot



Module 1
Processing & Chunking

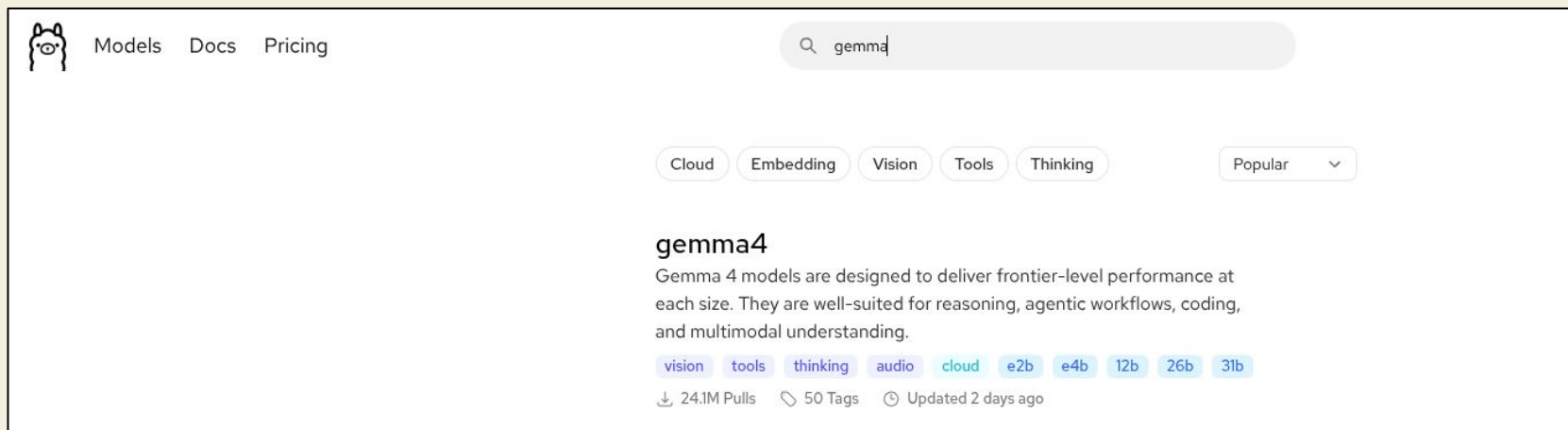
Module 2
Vector Stores

Module 3
Local LLMs and ChatApp

Module 4
Evaluation

Ollama and Local LLMs

- Ollama
 - A wrapper for large language models
 - Ollama is not an LLM
 - Ollama runs LLMs on your local machine
 - <https://ollama.com/download>
- Ollama website lists available LLMs



Local LLMs vs Cloud LLMs

Criteria	Local LLMs	Cloud LLMs
Data Privacy	✓	
Wifi Not Needed	✓	
Cost Predictability	✓	
Scalability		✓
Infrastructure Maintenance		✓

Caution: “Telemetry”

- Open source software is great
- Beware ‘telemetry’ settings: anonymized usage stats

==== Environment Variables ====

```
# Hugging Face
export HF_HUB_DISABLE_TELEMETRY=1
export DISABLE_TELEMETRY=1

# ChromaDB & Open-Source Utilities
export ANONYMIZED_TELEMETRY=False

# Streamlit
export STREAMLIT_BROWSER_GATHER_USAGE_STATS=false
```

==== Python Scripts ====

```
import os
os.environ["HF_HUB_DISABLE_TELEMETRY"] = "1"

import chromadb
from chromadb.config import Settings

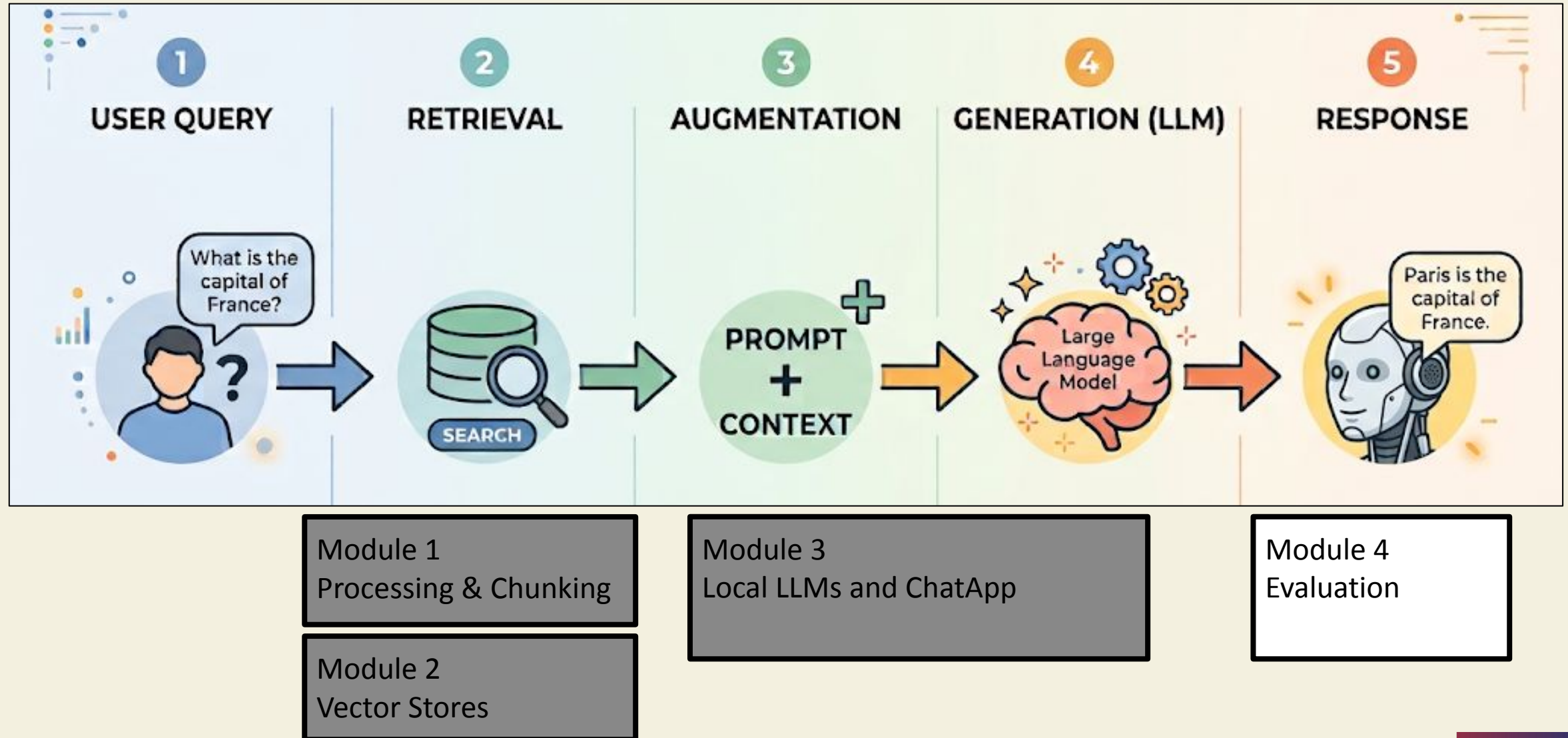
client = chromadb.PersistentClient(
    path="./chroma_db",
    settings=Settings(anonymized_telemetry=False)
)

# .streamlit/config.toml
[browser]
gatherUsageStats = false
```

Module 3: Hands on

- Begin!
 - module_03_retrieval_augmented_generation
 - [README.md](#)
 - Try: exercises
- Reconvene in 15 minutes
- Feel free to work ahead
- Questions? Ask:
 - Search engines & chatbots
 - Your neighbor
 - Me “Greg”

Module 4: RAG Bot Evaluation



RAG Evaluation

- How 'good' is the RAGbot?
- Evaluation
 - Avoid hallucinations
 - Answer accurately
- Approaches
 - Simple word overlap
 - LLM as a judge

Remember: if the right documents are not retrieved, the bot is not going to be able to answer correctly

Structured LLM Outputs with Pydantic

- Pydantic
 - A python library that enforces dictionary structure
 - Creates hard api contract for code and LLM outputs
- Zero hallucinated keys
- Bounded numeric scores
- Self-correcting parsing

Workshop Summary

- Retrieval Augmented Generation
 - A chatbot that leverages external knowledge
- Motivation: extensible knowledge store & data privacy
- RAG Pipeline
 - Preprocess
 - Embed
 - Retrieve
 - Generate answer & evaluate

Module 4: Hands on & Sendoff

- Begin!
 - module_04_evaluation
 - [README.md](#)
 - Try: exercises
- Feel free to work ahead (Bonus module 5)
- Questions? Ask:
 - Search engines & chatbots
 - Your neighbor
 - Me “Greg”: gregory.ciccarelli@sao.si.edu

Build responsibly!

Questions?

Implementing Secure, On-Premise
Retrieval Augmented Generation

Gregory Ciccarelli, Ph.D.
gregory.ciccarelli@sao.si.edu

WORKSHOP SURVEY: <https://forms.gle/tKnudkQg1QCZQ1SE7>